Laborator Python: funcții, metode și importuri pe web

Laborator pentru studenții din anul I

Nume și prenume: Sevostian Daniil

Grupa: SI-266 Data: 29.10.2026

Prezentarea laboratorului

În acest laborator veți scrie programe Python care comunică cu un site web real și veți exersa trei concepte de bază: **importurile**, **metodele** și **funcțiile**. La final veți avea propriul modul Python, care generează un raport despre un site web.

După acest laborator veți putea:

- Să importați module din biblioteca standard și module externe și să explicați diferența dintre ele.
- Să apelați metode și să citiți atribute ale obiectelor create de alt cod (răspunsuri, șiruri de caractere, dicționare).
- Să scrieți propriile funcții, cu parametri, valori implicite, valori returnate, docstring-uri și tratarea erorilor.
- Să împărțiți codul într-un modul propriu și să îl importați dintr-un alt fișier.

Ce trebuie să știți înainte: variabile, instrucțiuni `if`, bucle `for`, liste și dicționare.

Site-ul țintă: toate exercițiile de citire folosesc `https://cybercor.org`. Exercițiile care *trimit* date folosesc `https://httpbin.org`, un serviciu public creat pentru testarea cererilor.

Termeni web esențiali

Termen	Ce înseamnă	Exemplu
URL	Adresa unei resurse pe web	`https://cybercor.org/robots.txt`
Cerere (request)	Mesajul pe care programul vostru îl trimite unui server	`GET /`
Răspuns (response)	Răspunsul serverului la cerere	cod de stare + antete + corp
Cod de stare (status code)	Un număr care arată cum s-a încheiat cererea	`200` OK, `301` mutat, `404` negăsit
Antet (header)	O pereche nume–valoare cu informații suplimentare	`Content-Type: text/html`
Corp (body)	Conținutul propriu-zis returnat	codul HTML al paginii
HTML	Limbajul de marcare în care sunt scrise paginile web	`<title>CYBERCOR</title>`

Importuri, metode și funcții, pe scurt

- Un **import** aduce în programul vostru cod scris de altcineva: `import requests`.

- O **metodă** este o funcție care aparține unui obiect și se apelează cu punct: `response.json()`, `text.lower()`.
- O **funcție** este un bloc de cod cu nume, reutilizabil, pe care îl scrieți voi: `def get_title(html): ...`.

Configurare și reguli

Înainte de exercițiul 1, instalați o bibliotecă externă și creați un director pentru laborator.

1. Verificați versiunea Python cu `python --version` (3.9 sau mai nouă).
2. Instalați biblioteca `requests`: `pip install requests`.
3. Creați un director numit `web_lab` și lucrați în el. Salvați fiecare parte într-un fișier separat: `part1_imports.py`, `part2_methods.py` și așa mai departe.
4. Începeți fiecare fișier cu acest șablon:

```
# Laborator: funcții, metode și importuri pe web
# Student: Sevostian-Daniil

BASE_URL = "https://cybercor.org"
ECHO_URL = "https://httpbin.org"
TIMEOUT = 10  # secunde
```

Reguli de lucru

Lucrați cu un site web real, de care depind oameni reali. Respectați aceste reguli la fiecare exercițiu:

- **Folosiți mereu un timeout.** Fiecare cerere primește `timeout=TIMEOUT`, ca programul să nu se blocheze la nesfârșit.
- **Faceți pauze între cereri.** Când trimiteți mai multe cereri una după alta, puneți `time.sleep(1)` între ele.
- **Doar citire.** Către cybercor.org trimiteți doar cereri `GET` și `HEAD`. Tot ce trimite date (`POST`, parametri de interogare, antete personalizate pe care vreți să le verificați) merge către httpbin.org.
- **Nicio buclă fără limită.** Nu scrieți niciodată o buclă `while True:` care trimite cereri.
- **Rămâneți pe țintă.** Accesați doar URL-urile indicate în exerciții. Nu scanați alte site-uri.

Încălcarea acestor reguli poate duce la blocarea adresei IP a întregii săli, de aceea respectarea lor face parte din notă.

Partea 1: Importuri (exercițiile 1–8)

Scop: învățați diferitele moduri de a aduce cod în program. Fișier: `part1_imports.py`.

Exercițiul 1. **Primul import.** Scrieți `import requests` și afișați `requests.__version__`. Apoi scrieți `import urllib.request`. Într-un comentariu, explicați de ce `requests` a avut nevoie de `pip install`, iar `urllib` nu.

requests este o bibliotecă externă și trebuie instalată cu pip install requests.
urllib face parte din biblioteca standard Python și nu necesită instalare separată.

Exercițiul 2. **Două stiluri de import.** Descărcați pagina `BASE_URL` cu `requests.get()` și afișați codul de stare. Apoi rescrieți același script folosind `from requests import get`. Într-un comentariu, scrieți câte un avantaj al fiecărui stil.

requests.get este mai clar deoarece se vede modulul folosit.
from requests import get este mai scurt, deoarece putem apela direct funcția get().

Exercițiul 3. Alias-uri. Importați requests cu un alias: `import requests as rq`. Descărcați pagina principală cu `rq.get()`. Într-un comentariu, spuneți când un alias face codul mai ușor de citit și când îl face mai greu de citit.

Alias-ul face codul mai ușor de citit când numele este scurt și folosit des.
Un alias poate face codul mai greu de citit dacă numele ales nu este clar.

Exercițiul 4. Doar biblioteca standard. Descărcați pagina principală folosind doar `urllib.request.urlopen()`. Afișați atributul `.status` al răspunsului și primele 200 de caractere din corp.

> **Indiciu:** `.read()` returnează octeți (bytes). Folosiți `.decode("utf-8")` ca să îi transformați într-un șir de caractere.

urllib.request.Request creează o cerere către site și adaugă un User-Agent.
urlopen() trimite cererea și primește răspunsul de la server.
.status afișează codul de stare HTTP.
.read() citește conținutul paginii în bytes, iar .decode("utf-8") îl transformă în text.
body[:200] afișează primele 200 de caractere din pagina HTML.

Exercițiul 5. Priviți în interiorul unui modul. Rulați `print(dir(requests))`. Alegeți trei nume din rezultat și scrieți, într-un comentariu, dacă fiecare este o funcție, o clasă sau un modul.

dir(requests) afișează toate numele disponibile în modulul requests,
cum ar fi funcții, clase și alte module.
get - funcție, este folosită pentru a trimite cereri HTTP de tip GET.
Session - clasă, este folosită pentru a crea și gestiona o sesiune HTTP.
exceptions - modul, conține excepțiile și erorile folosite de requests.

Exercițiul 6. Citiți documentația. Rulați `help(requests.get)`. Găsiți parametrul care setează timpul maxim de așteptare și folosiți-l într-o cerere.

help(requests.get) afișează documentația funcției get().
Parametrul timeout stabilește timpul maxim de așteptare pentru cerere.
În acest caz, TIMEOUT = 10, deci cererea are un timeout de 10 secunde.

Exercițiul 7. Cronometrarea unei cereri. Importați `time` și măsurați cât durează o cerere către pagina principală, apelând `time.perf_counter()` înainte și după. Comparați rezultatul cu `response.elapsed`.

time.perf_counter() măsoară timpul înainte și după efectuarea cererii.
Diferența dintre end și start reprezintă timpul total al cererii.
response.elapsed arată timpul măsurat de biblioteca requests pentru primirea răspunsului.
Valorile pot fi apropiate, dar nu trebuie să fie exact egale.

Exercițiul 8. Când un import eșuează. Scrieți un script care încearcă `import bs4` într-un bloc `try` / `except ImportError`. Dacă modulul lipsește, afișați: `Instalați modulul cu: pip install beautifulsoup4`.

Blocul try încearcă să importe modulul bs4.
Dacă modulul nu este instalat, apare ImportError și se execută blocul except.
Astfel, programul nu se oprește cu eroare și afișează comanda necesară pentru instalare.
Modulul este un singur fișier Python care conține cod reutilizabil.

Pachetul este o colecție de module Python organizate împreună.
Biblioteca este o colecție mai mare de module și pachete folosite pentru anumite sarcini.

> **Verificați-vă:** care este diferența dintre un modul, un pachet și o bibliotecă?

Partea 2: Metode și atribute ale obiectelor web (exercițiile 9–20)

Scop: învățați să folosiți instrumentele pe care obiectele le au deja. Un **atribut** este o valoare (`response.status_code`); o **metodă** este o acțiune pe care o apelați cu paranteze (`response.json()`).
Fișier: `part2_methods.py`.

Exercițiul 9. Elementele de bază ale răspunsului. Descărcați pagina principală și afișați `response.status_code`, `response.ok`, `response.url` și `response.encoding`. Lângă fiecare linie, scrieți un comentariu care spune dacă este un atribut sau o metodă.

response reprezintă răspunsul primit de la server.
status_code conține codul de stare HTTP al răspunsului.
ok indică dacă cererea s-a încheiat cu succes.
url conține adresa finală a paginii.
encoding indică modul de codificare a textului răspunsului.
Toate cele patru sunt atribute, deoarece sunt accesate fără paranteze.

Exercițiul 10. Tratarea erorilor cu o metodă. Apelați `response.raise_for_status()` pe pagina principală. Apoi trimiteți o cerere către `BASE_URL + "/this-page-does-not-exist"` și prindeți excepția `requests.HTTPError`. Afișați un mesaj prietenos în loc să lăsați programul să se oprească cu eroare.

raise_for_status() verifică dacă răspunsul conține o eroare HTTP.
Pagina principală returnează codul 200, deci nu apare nicio eroare.
În prezent, serverul cybercor.org returnează codul 200 chiar și pentru
calea /this-page-does-not-exist, de aceea HTTPError nu este generată.
Dacă serverul ar returna 404, excepția ar fi prinsă de blocul except.

Exercițiul 11. Toate antetele. Parcurgeți `response.headers.items()` și afișați fiecare antet în forma `Nume: valoare`.

response.headers conține antetele HTTP primite de la server.
Metoda .items() permite parcurgerea fiecărui antet sub forma
unei perechi: nume și valoare.
Bucla for afișează fiecare antet în forma "Nume: valoare".

Exercițiul 12. Metode de dicționar. Folosiți `response.headers.get()` pentru a afișa antetele `Server` și `Content-Type`, cu `"lipsește"` ca valoare implicită. Apoi încercați `response.headers.get("content-type")`, cu litere mici. Ce observați?

Metoda .get() caută valoarea unui antet după numele acestuia.
Dacă antetul nu există, este returnată valoarea implicită "lipsește".
În obiectul response.headers, numele antetelor nu sunt sensibile
la litere mari sau mici.
De aceea, "Content-Type" și "content-type" returnează aceeași valoare.

Exercițiul 13. Înlănțuirea metodelor de șir. Numărați de câte ori apare cuvântul `cyber` pe pagina principală, cu `response.text.lower().count("cyber")`. Într-un comentariu, explicați de ce puteți înlănțui `.lower()` și `.count()`.

response.text conține codul HTML al paginii sub formă de șir de caractere.
Metoda .lower() transformă toate literele din text în litere mici.
Metoda .count("cyber") numără de câte ori apare cuvântul "cyber".
Metodele pot fi înlănțuite deoarece .lower() returnează tot un șir de caractere, iar metoda .count() poate fi apelată pe acest șir.

Exercițiul 14. Găsirea titlului. Folosiți `.find()` și felierea (slicing) pe `response.text` pentru a extrage textul dintre `<title>` și `</title>`. Curățați-l cu `.strip()` și afișați-l.

> **Indiciu:** `start = html.find("<title>") + len("<title>")`

response.text conține codul HTML al paginii.
.find("<title>") găsește poziția unde începe tag-ul <title>.
len("<title>") este adăugat pentru a începe extragerea după tag.
.find("</title>") găsește poziția unde se termină titlul.
Prin slicing [start:end] extragem textul dintre cele două tag-uri.
.strip() elimină spațiile inutile de la începutul și sfârșitul titlului.

Exercițiul 15. Numărarea liniilor. Împărțiți codul HTML în linii cu `.splitlines()`. Afișați câte linii sunt și lungimea celei mai lungi linii, folosind `max(lines, key=len)`.

.splitlines() împarte codul HTML în linii și returnează o listă.
len(lines) arată numărul total de linii din pagina HTML.
max(lines, key=len) găsește linia care are cea mai mare lungime.
len(longest_line) afișează numărul de caractere din cea mai lungă linie.

Exercițiul 16. Verificarea HTTPS. Folosiți `.startswith()` pentru a verifica dacă `response.url` începe cu `https://`. Afișați `Conexiune securizată` sau `Conexiune nesecurizată`.

response.url conține adresa finală a paginii.
Metoda .startswith("https://") verifică dacă URL-ul începe cu "https://".
Dacă rezultatul este True, conexiunea folosește HTTPS și este afișat mesajul "Conexiune securizată".
În caz contrar, este afișat mesajul "Conexiune nesecurizată".

Exercițiul 17. Urmărirea redirecționărilor. Trimiteți o cerere către `http://cybercor.org` (HTTP simplu, fără S). Parcurgeți `response.history` și afișați codul de stare și URL-ul fiecărei redirecționări, apoi URL-ul final.

Cererea este trimisă inițial către versiunea HTTP a site-ului.
response.history conține lista redirecționărilor efectuate până la ajungerea la pagina finală.
Pentru fiecare redirecționare sunt afișate codul de stare și URL-ul.
response_redirect.url conține adresa finală după toate redirecționările.

Exercițiul 18. HEAD versus GET. Trimiteți o cerere către pagina principală cu `requests.head()` și una cu `requests.get()`. Comparați `len(response.content)` pentru ambele și explicați diferența într-un comentariu.

Cererea HEAD solicită doar informațiile și antetele răspunsului, fără a descărca corpul paginii.
Din acest motiv, response_head.content este de obicei gol.
Cererea GET descarcă atât antetele, cât și conținutul paginii.
De aceea, lungimea conținutului GET este mai mare decât la HEAD.

Exercițiul 19. Cookie-uri. Parcurgeți `response.cookies` și afișați, pentru fiecare cookie, atributele `.name` și `.secure`. Dacă nu există cookie-uri, afișați `Niciun cookie setat`.

response.cookies conține cookie-urile primite de la server.
Dacă există cookie-uri, bucla for le parcurge pe fiecare.
Atributul .name conține numele cookie-ului.
Atributul .secure indică dacă acel cookie este transmis doar prin HTTPS.
Dacă serverul nu a setat niciun cookie, este afișat mesajul
"Niciun cookie setat".

Exercițiul 20. Sesiuni. Creați un obiect `requests.Session()` și setați un `User-Agent` personalizat cu `session.headers.update({"User-Agent": "WebLab-Sevostian-Daniil"})`. Trimiteți o cerere către `ECHO_URL + "/headers"` și confirmați în rezultat că antetul a fost trimis.

requests.Session() creează o sesiune care poate păstra aceleași
setări pentru mai multe cereri.
session.headers.update() adaugă un antet User-Agent personalizat.
Cererea este trimisă către httpbin.org/headers, care returnează
antetele primite de server.
Metoda .json() transformă răspunsul JSON într-un dicționar Python.
La final verificăm dacă User-Agent-ul personalizat a fost trimis corect.
response.text este un atribut și conține direct textul răspunsului,
de aceea este accesat fără paranteze.
response.json() este o metodă care execută o acțiune și transformă
răspunsul JSON într-un obiect Python, de aceea se apelează cu paranteze.

> **Verificați-vă:** atât `response.text`, cât și `response.json()` vă dau corpul răspunsului. De ce unul are paranteze, iar celălalt nu?

Partea 3: Scrierea propriilor funcții (exercițiile 21–34)

Scop: organizați codul în funcții reutilizabile. De acum înainte, fiecare exercițiu este o funcție pe care o apelați cel puțin o dată. Fișier: `part3_functions.py`.

```
def get_status(url):
    """Returnează codul de stare HTTP pentru adresa url."""
    response = requests.get(url, timeout=TIMEOUT)
    return response.status_code
```

Exercițiul 21. Prima funcție. Scrieți `fetch(url)`, care returnează obiectul răspuns. Apelați-o pentru `BASE_URL` și afișați codul de stare.

fetch() primește adresa ca parametru și returnează întregul răspuns.
Din obiectul returnat putem citi status_code, text și headers.

Exercițiul 22. Returnarea unei singure valori. Scrieți `get_status(url)`, care returnează doar codul de stare. Apelați-o pentru `/`, `/robots.txt` și `/sitemap.xml`, cu `time.sleep(1)` între apeluri.

get_status() returnează un int, spre deosebire de fetch().
Funcția pause_between_requests() aplică time.sleep(1) între apeluri.

Exercițiul 23. Parametri impliciți. Modificați funcția în `fetch(url, timeout=10)`. Apelați-o o dată cu valoarea implicită și o dată cu `timeout=3`.

Noua definiție înlocuiește funcția fetch() din exercițiul 21.
Fără argumentul timeout se folosesc 10 secunde; timeout=3 schimbă valoarea.

Exercițiul 24. O singură sarcină pe funcție. Scrieți `get_title(html)`, care primește un șir HTML și returnează titlul paginii, refolosind logica din exercițiul 14. Observați că această funcție nu face nicio operație de rețea: descărcarea și analiza sunt acum separate.

.find() găsește tag-urile, slicing extrage titlul, iar .strip() îl curăță.
Funcția primește HTML deja descărcat și nu efectuează cereri de rețea.

Exercițiul 25. Docstring-uri. Adăugați un docstring funcției `get_title` și afișați-l cu `help(get_title)`.

Docstring-ul este textul dintre triple ghilimele de la începutul funcției.
help(get_title) afișează semnătura și explicația scrisă în docstring.

Exercițiul 26. Adnotări de tip. Adăugați adnotări de tip (type hints) funcțiilor, de exemplu `def get_status(url: str) -> int:`. Apelați `get_status(123)` și observați ce se întâmplă. Vă opresc adnotările de tip?

Adnotările str și int documentează tipurile și ajută editorul.
Python permite apelul get_status(123); requests respinge URL-ul fără schemă.
Eroarea vine din biblioteca requests, nu din verificarea adnotărilor.

Exercițiul 27. Fără opriri neașteptate. Scrieți `page_exists(url)`, care returnează `True` sau `False`. Folosiți `try` / `except requests.RequestException`, astfel încât funcția să returneze `False` în loc să se oprească cu eroare, chiar și pentru un URL precum `https://this-domain-does-not-exist.invalid`.

try / except împiedică oprirea programului la o eroare de conexiune.
.ok verifică succesul HTTP, dar un site poate răspunde 200 și pentru o cale absentă.

Exercițiul 28. Returnarea unui dicționar. Scrieți `check_paths(base, paths)`, care primește o listă precum `["/", "/robots.txt", "/sitemap.xml"]` și returnează un dicționar `{path: status_code}`.

Dicționarul asociază fiecare cale cu răspunsul serverului.
Pauzele se aplică în fetch(), inclusiv pentru cererile din buclă.

Exercițiul 29. Argumente cu nume. Scrieți `get_header(url, name, default="lipsește")`, care returnează valoarea unui singur antet. Apelați-o cu argumente cu nume: `get_header(BASE_URL, name="Server")`.

name="Server" este un argument cu nume, deci sensul său este vizibil în apel.
.get() evită KeyError atunci când antetul nu există.

Exercițiul 30. Antete de securitate. Scrieți `security_headers(url)`, care verifică prezența acestor cinci antete și returnează `{antet: True/False}`:

- `Strict-Transport-Security`
- `Content-Security-Policy`
- `X-Frame-Options`
- `X-Content-Type-Options`
- `Referrer-Policy`

Operatorul in verifică dacă fiecare nume se află în antetele răspunsului.
Verificăm doar prezența antetelor, nu corectitudinea configurației lor.

Exercițiul 31. Funcții care apelează funcții. Scrieți `score_headers(results)`, care primește dicționarul din exercițiul 30 și returnează un șir precum `"3/5"`. Apelați-le împreună: `score_headers(security_headers(BASE_URL))`.

True este numărat ca 1, iar False ca 0 de către sum().
Rezultatul security_headers() devine argument pentru score_headers().

Exercițiul 32. Două funcții, o sarcină. Scrieți `fetch_robots(base)`, care returnează textul fișierului `/robots.txt` sau `None` dacă fișierul lipsește. Apoi scrieți `disallowed_paths(robots_text)`, care returnează o listă cu toate valorile `Disallow:`. Asigurați-vă că funcționează și atunci când primește `None`.

Descărcarea și analiza sunt separate în două funcții.
Disallow fără valoare nu interzice o cale, deci este ignorat.
Nu accesăm căile găsite în robots.txt.

Exercițiul 33. Oricâte argumente. Scrieți `response_times(*urls)`, care returnează un dicționar `{url: secunde}`. Apelați-o cu două URL-uri.

*urls colectează oricâte argumente poziționale într-un tuplu.
perf_counter() măsoară durata cererii; pauza este făcută înainte de cronometrare.

Exercițiul 34. Argumente cu nume colectate. Scrieți `log(message, **details)`, care afișează mesajul urmat de fiecare detaliu. Exemplu: `log("verificat", url=BASE_URL, status=200)` afișează `verificat | url=https://cybercor.org | status=200`.

**details colectează argumentele cu nume într-un dicționar.
print() doar afișează o valoare; return o trimite înapoi apelantului.
O valoare returnată poate fi salvată sau transmisă unei alte funcții.

> **Verificați-vă:** care este diferența dintre a afișa o valoare cu `print()` în interiorul unei funcții și a o returna cu `return`?

Partea 4: Alte module din biblioteca standard (exercițiile 35–43)

Scop: combinați mai multe importuri pentru a rezolva probleme mai mari. Puneți fiecare soluție într-o funcție. Fișier: `part4_modules.py`.

Modul	La ce îl folosiți aici
`urllib.parse`	Descompunerea și compunerea URL-urilor
`re`	Căutarea de tipare în text
`html.parser`	Citirea corectă a codului HTML
`hashlib`	Calcularea amprentei unei pagini
`json`	Salvarea datelor într-un fișier
`socket`	Aflarea adreselor IP și deschiderea conexiunilor

Modul	La ce îl folosiți aici
`ssl` + `datetime`	Citirea datei de expirare a unui certificat

Exercițiul 35. Descompunerea unui URL. Folosiți `urllib.parse.urlparse()` pe `https://cybercor.org/path?x=1#top`. Afișați `.scheme`, `.netloc`, `.path`, `.query` și `.fragment`.

urlparse() descompune adresa în componente accesibile ca atribute.
query conține x=1, iar fragment conține top, fără semnele ? și #.

Exercițiul 36. Compunerea URL-urilor. Folosiți `urllib.parse.urljoin()` pentru a transforma legăturile relative `/about`, `contact.html` și `../index.html` în URL-uri complete, pornind de la `BASE_URL`.

urljoin() combină baza și legătura relativă, inclusiv segmentele ../.
Construim adresele cerute fără să accesăm aceste pagini.

Exercițiul 37. Extragerea legăturilor. Folosiți `re.findall(r'href="([^"]+)"', html)` pentru a obține toate legăturile de pe pagina principală. Puneți codul într-o funcție `extract_links(html)`, care returnează legăturile fără duplicate.

re.findall(r'href="([^"]+)"', html) extrage valorile href.
dict.fromkeys() elimină duplicatele și păstrează ordinea lor.

Exercițiul 38. Interne sau externe? Scrieți `split_links(links, domain)`, care returnează două liste: legăturile de pe același domeniu și legăturile către alte site-uri. Folosiți mai întâi `urljoin()`, apoi `urlparse().netloc`.

urljoin() transformă fiecare legătură într-un URL absolut.
urlparse().netloc permite compararea domeniilor; nu accesăm legăturile.
Schemele mailto:, tel: și javascript: nu sunt numărate ca pagini web.

Exercițiul 39. Propriile metode. Creați o clasă care moștenește `html.parser.HTMLParser` și suprascrieți metoda `handle_starttag()`, pentru a colecta atributul `src` al fiecărui tag `<img>`.

```
from html.parser import HTMLParser

class ImageFinder(HTMLParser):
    def __init__(self):
        super().__init__()
        self.images = []

    def handle_starttag(self, tag, attrs):
        # codul vostru aici
        pass
```

> **Indiciu:** `attrs` este o listă de perechi `(nume, valoare)`, de exemplu `[("src", "/logo.png"), ("alt", "Logo")]`. Expresia `dict(attrs).get("src")` vă dă valoarea atributului `src` sau `None`.

> **Cum testați exercițiul 39.** Testați mai întâi parserul pe un fragment HTML scris de voi, pentru care știți dinainte rezultatul corect. Abia după aceea rulați-l pe site-ul real. Adăugați acest test sub clasa voastră:

```
test_html = """
<html><body>
  <img src="/logo.png" alt="Logo">
  <IMG SRC="poza.jpg">
  <img alt="imagine fără src">
  <img src="https://cdn.example.com/banner.webp" />
  <a href="/despre">Aceasta nu este o imagine</a>
</body></html>
"""

finder = ImageFinder()
finder.feed(test_html)
print(finder.images)

assert finder.images == [
    "/logo.png",                          # imagine obișnuită
    "poza.jpg",                           # tag scris cu majuscule
    "https://cdn.example.com/banner.webp",  # tag care se închide singur
], "Parserul nu a găsit exact imaginile așteptate"
print("Testul a trecut!")
```

Dacă testul trece, parserul vostru: găsește tag-urile `<img>` indiferent de majuscule, ignoră tag-urile `<img>` fără `src`, ignoră celelalte tag-uri (de exemplu `<a>`) și păstrează ordinea din pagină. Dacă `assert` eșuează, comparați lista afișată de `print()` cu lista așteptată.

Apoi rulați parserul pe pagina reală:

```
response = requests.get(BASE_URL, timeout=TIMEOUT)
finder = ImageFinder()
finder.feed(response.text)
print(len(finder.images), "imagini găsite")
for src in finder.images:
    print(src)
```

Verificați rezultatul în două moduri: comparați numărul găsit cu `response.text.lower().count("<img")` și deschideți pagina în browser, apoi apăsați Ctrl+U ca să vedeți codul sursă. O listă goală nu înseamnă neapărat o greșeală: unele site-uri afișează imaginile prin CSS sau SVG, nu prin tag-uri `<img>`. Tocmai de aceea testul cu HTML cunoscut vine primul.

HTMLParser apelează automat handle_starttag() când feed() întâlnește un tag.
Parserul normalizează tag-urile și numele atributelor scrise cu majuscule.
O listă goală poate fi corectă dacă imaginile sunt afișate prin CSS sau SVG.
Numărul tag-urilor poate fi mai mare dacă unele imagini nu au src.

Exercițiul 40. Amprenta paginii. Scrieți `page_fingerprint(url)`, care returnează `hashlib.sha256(response.content).hexdigest()`. Rulați-o de două ori și comparați rezultatele. Când ar fi diferite?

page_fingerprint() folosește hashlib.sha256(response.content).
Amprentele diferă dacă se modifică oricare octet, inclusiv conținutul dinamic.
fetch() pune o pauză între cele două cereri.

Exercițiul 41. Salvarea în JSON. Salvați antetele paginii principale în `headers.json` cu `json.dump(dict(response.headers), file, indent=2)`. Apoi încărcați fișierul înapoi cu `json.load()` și afișați un antet din el.

json.dump() scrie dicționarul în fișier, iar json.load() îl reconstruiește.
Fișierul este salvat lângă program, indiferent de directorul terminalului.
În JSON cheile sunt obișnuite; CaseInsensitiveDict permite căutarea antetelor.

Exercițiul 42. Interogare DNS. Scrieți `resolve(hostname)`, care returnează `socket.gethostbyname(hostname)`. Afișați adresa IP a domeniului `cybercor.org`.

resolve() folosește socket.gethostbyname(hostname).
Adresa IP este obținută prin DNS și poate fi diferită în timp.

Exercițiul 43. Expirarea certificatului. Scrieți `cert_days_left(hostname)`, care se conectează la portul 443, citește certificatul și returnează câte zile mai sunt până la expirarea lui.

> **Indiciu:** folosiți `ssl.create_default_context()`, `socket.create_connection((hostname, 443))`, `context.wrap_socket(sock, server_hostname=hostname)` și `getpeercert()["notAfter"]`. Convertiți data cu `ssl.cert_time_to_seconds()`.

cert_days_left() folosește ssl.create_default_context() și conexiunea la 443.
wrap_socket() validează certificatul pentru server_hostname.
getpeercert()["notAfter"] este convertit cu ssl.cert_time_to_seconds().
Diferența se calculează față de momentul curent în UTC, cu timeout de 10 secunde.

> **Verificați-vă:** în exercițiul 39, cine apelează `handle_starttag()`? Voi nu o apelați niciodată direct.

Partea 5: Construirea propriului modul (exercițiile 44–50)

Scop: transformați funcțiile într-un modul pe care alte programe îl pot importa, la fel ca `requests`. La final, directorul vostru ar trebui să arate astfel:

```
web_lab/
├── webtools.py     # modulul vostru: toate funcțiile reutilizabile
├── main.py         # programul care importă webtools
├── report.csv      # creat în exercițiul 49
└── report.json     # creat în exercițiul 50
```

Exercițiul 44. Crearea unui modul. Mutați `fetch`, `get_status`, `get_title` și `security_headers` într-un fișier nou, `webtools.py`. În `main.py`, scrieți `import webtools` și apelați `webtools.get_title(webtools.fetch(BASE_URL).text)`.

import webtools permite apeluri în forma webtools.nume_funcție().
Funcția este apelată din main() cu URL-ul primit din linia de comandă.

Exercițiul 45. Importarea anumitor nume. În `main.py`, treceți la `from webtools import get_title, security_headers`. Într-un comentariu, explicați ce s-ar întâmpla dacă `main.py` ar avea și o funcție proprie numită `get_title`.

from webtools import ... permite apelarea fără prefixul webtools.
Dacă definim după import o funcție proprie get_title, ea înlocuiește numele importat.

Dacă importul vine după definiția proprie, numele importat o înlocuiește pe aceasta.
webtools.get_title rămâne accesibil prin numele modulului în ambele cazuri.

Exercițiul 46. Blocul main. Adăugați acest bloc la finalul fișierului `webtools.py`:

```
if __name__ == "__main__":
    print("Autotest:", get_status("https://cybercor.org"))
```

Rulați `python webtools.py`, apoi `python main.py`. Explicați de ce autotestul rulează doar în primul caz.

Blocul de autotest se află la finalul webtools.py.
python webtools.py îl execută deoarece __name__ este "__main__".
python main.py importă modulul cu __name__ == "webtools" și nu execută autotestul.

Exercițiul 47. Constante de modul. Adăugați constanta `DEFAULT_HEADERS = {"User-Agent": "WebLab-Sevostian-Daniil"}` în `webtools.py` și folosiți-o în `fetch()`. În `main.py`, importați constanta și afișați-o.

Constanta este definită o singură dată în webtools și reutilizată la import.
fetch() trimite acest User-Agent către httpbin, conform regulilor laboratorului.

Exercițiul 48. Argumente din linia de comandă. Folosiți `argparse` în `main.py`, astfel încât programul să ruleze ca `python main.py https://cybercor.org`. Transmiteți argumentul URL funcțiilor voastre, în loc să folosiți `BASE_URL`.

argparse citește argumentul URL, iar main() îl transmite funcțiilor.
Exemplu: python main.py https://cybercor.org
Dacă argumentul lipsește, este folosit BASE_URL.

Exercițiul 49. Un raport CSV. Folosiți modulul `csv` pentru a salva rezultatul funcției `check_paths()` în `report.csv`, cu coloanele `path`, `status` și `checked_at`. Pentru oră, folosiți `datetime.now().isoformat()`.

DictWriter scrie câte un rând pentru fiecare cale verificată.
newline="" evită liniile goale suplimentare în CSV pe Windows.
datetime.now().isoformat() scrie data și ora în format ISO.

Exercițiul 50. Proiect final: raportul site-ului. Scrieți `site_report(url)` în `webtools.py`. Funcția trebuie să folosească funcțiile voastre pentru a colecta toate informațiile de mai jos, să le afișeze și să le salveze în `report.json`:

1. Codul de stare și URL-ul final
2. Titlul paginii
3. Adresa IP
4. Lanțul de redirecționări pornind de la `http://`
5. Scorul antetelor de securitate, de exemplu `3/5`
6. Zilele rămase până la expirarea certificatului
7. Numărul de legături interne și externe
8. Căile interzise din `robots.txt`

Un exemplu de rezultat (valorile voastre vor fi diferite):

```
=== Raport site: https://cybercor.org ===
Cod de stare:      200
Titlu:             CYBERCOR
Adresă IP:         203.0.113.10
Redirecționări:    301 -> https://cybercor.org/
Scor securitate:   3/5
Certificat:        74 de zile rămase
Legături:          38 interne, 12 externe
Căi interzise:     /admin, /private
Salvat în report.json
```

site_report() refolosește funcțiile pentru descărcare, titlu, DNS,
redirecționări, antete, certificat, legături și robots.txt.
Erorile reale sunt salvate separat; o valoare necunoscută devine null în JSON.
Legăturile și căile Disallow sunt analizate, fără a fi accesate.

> **Bonus:** instalați `beautifulsoup4` și rescrieți `get_title()` și `extract_links()` folosind această bibliotecă. Comparați cât de mult mai scurte și mai sigure devin.

BeautifulSoup gestionează atributele, ghilimelele simple și entitățile HTML.
Varianta cu find() presupune un tag <title> simplu; regex-ul caută doar ghilimele duble.
Parserul reduce verificările manuale și este mai sigur pentru HTML variat.

Predare și notare

Predați întregul director web_lab ca o singură arhivă .zip, denumită după regula disciplinei:

`L4_TP_Nume_Prenume_Grupa.zip`

Exemplu, pentru studentul Popescu Ion din grupa SI-266: **`L4_TP_Popescu_Ion_SI-266.zip`**

Fără spații și fără diacritice în denumire. Grupa se scrie cu majuscule și cu cratimă: SI-265 sau SI-266.

	Regula
Adresa de email	cristian.gonceari@isa.utm.md
Subiectul emailului	L4_TP_Popescu_Ion_SI-266
	adică exact numele arhivei, fără „.zip”. Nimic altceva în subiect.
Atașament	o singură arhivă .zip, care conține directorul web_lab
Termen limită	_____

De ce contează subiectul: cu acest subiect, toate lucrările unui laborator pot fi găsite și sortate instantaneu în căsuța poștală. Un subiect scris liber înseamnă că lucrarea ta se poate pierde printre celelalte.

Înainte de predare

- ☐ Fiecare fișier rulează fără erori de la început până la sfârșit.
- ☐ Fiecare cerere folosește un timeout, iar între cererile repetate există `time.sleep(1)`.
- ☐ Fiecare funcție, începând cu Partea 3, are un docstring.
- ☐ Toate cerințele de tip "explicați într-un comentariu" au răspuns în cod.
- ☐ `python main.py https://cybercor.org` afișează raportul complet și creează `report.json`.

- ☐ Directorul conține `report.csv` și `report.json`.
- ☐ Arhiva se numește L4_TP_Nume_Prenume_Grupa.zip, fără spații și fără diacritice.
- ☐ Subiectul emailului este identic cu numele arhivei, fără „.zip”.

Notare

Partea	Exerciții	Puncte
Partea 1: Importuri	1–8	10
Partea 2: Metode și atribute	9–20	20
Partea 3: Propriile funcții	21–34	25
Partea 4: Module din biblioteca standard	35–43	20
Partea 5: Propriul modul și proiectul final	44–50	20
Respectarea regulilor de lucru	toate	5
Total		**100**

Cum se transformă punctajul în notă

Punctaj	Nota
95 – 105	10
85 – 94	9
75 – 84	8
65 – 74	7
55 – 64	6
45 – 54	5
sub 45	nepromovat — lucrarea se reface

Exercițiul bonus (BeautifulSoup): până la 5 puncte în plus.